UNIVERSIDAD GERARDO BARRIOS

Facultad de Ciencias y Tecnología

Ingeniería en Sistemas y Redes Informáticas

SafeSite

Sistema de monitoreo de Equipo de Protección Personal (EPP) en tiempo real para el sector construcción

INTEGRANTES

Nombre completo	Carnet
César Alejandro Escobar Recinos	SMSS042725
Josue Rafael Escamilla Ventura	SMSS032825
Miguel Angel Arevalo Cruz	SMSS030825
Genesis Concepcion Noble Olivas	SMSS043825
Fernando Vladimir Obispo Gómez	smss024225

Asignatura	Docente	Fecha
Porgramación Computacional	Willian Alexis Montes Girón]	Noviembre de 2026

Índice

1. Introducción 2

2. Objetivo general y objetivos específicos 2

2.1 Objetivo general 2

2.2 Objetivos específicos 3

3. Delimitación del proyecto 3

3.1 Alcance 3

3.2 Fuera del alcance 3

3.3 Delimitación técnica, espacial y temporal 4

3.4 Limitaciones conocidas 4

4. Justificación de las tecnologías a utilizar 4

5. Requerimientos del proyecto 5

5.1 Requerimientos funcionales 5

5.2 Requerimientos no funcionales 6

1. Introducción

En la industria de la construcción, las personas que trabajan en obra están expuestas a riesgos como la caída de objetos, los golpes en la cabeza, el contacto con maquinaria y la baja visibilidad en zonas de tránsito. Dos de los elementos de protección más básicos para reducir estos riesgos son el **casco** y el **chaleco reflectivo**. Aun así, su uso depende casi por completo de la supervisión humana.

El problema. Un supervisor no puede observar todos los frentes de trabajo al mismo tiempo, por lo que el incumplimiento del uso de EPP suele pasar desapercibido hasta que ocurre un incidente o una inspección. Además, rara vez queda un registro objetivo de cuándo y con qué frecuencia se incumplió la norma, lo que dificulta analizar tendencias, exigir responsabilidades y mejorar de forma continua las medidas de seguridad. Las soluciones comerciales de videovigilancia con analítica suelen requerir equipos costosos, servidores con tarjeta gráfica o servicios en la nube, lo que las deja fuera del alcance de pequeñas y medianas constructoras.

La solución propuesta. SafeSite es una aplicación de escritorio que usa una cámara web y visión por computadora para detectar personas en tiempo real y comprobar si llevan casco y chaleco. Cuando detecta una infracción, avisa de inmediato con una alerta visual (recuadro rojo con el texto "ALERTA: FALTA EPP") y una alarma sonora continua, y conserva evidencia de cada caso: una fotografía, la fecha y hora, y el tipo de falta, guardados en una base de datos local. El sistema está pensado para ejecutarse en computadoras comunes sin tarjeta gráfica dedicada y procesa todo de forma local, sin enviar imágenes a internet.

Este documento describe los objetivos, la delimitación, la justificación de las tecnologías y los requerimientos del proyecto. Junto a él se entrega un prototipo inicial ejecutable que implementa la funcionalidad central: monitoreo en vivo con detección, alerta sonora y registro de infracciones.

2. Objetivo general y objetivos específicos

2.1 Objetivo general

Desarrollar un sistema de escritorio de monitoreo en tiempo real que, mediante visión por computadora aplicada al video de una cámara web, detecte a las personas y verifique el uso de casco y chaleco, emita alertas inmediatas y conserve evidencia de cada infracción, funcionando en equipos sin tarjeta gráfica dedicada.

2.2 Objetivos específicos

1. Capturar y procesar video en tiempo real desde una cámara web a 640 x 480 píxeles, aplicando optimizaciones para CPU (modelo liviano YOLOv8n, imágenes de entrada de 320 o 416 píxeles y salto de fotogramas).
2. Detectar personas con YOLOv8n y verificar la presencia de casco y chaleco mediante reglas sobre la imagen, con la posibilidad de sustituirlas por un modelo entrenado específicamente en EPP.
3. Notificar cada infracción con un recuadro rojo y el texto "ALERTA: FALTA EPP" y con una alarma sonora continua que se detenga cuando el equipo se regularice; marcar en verde "EPP COMPLETO" cuando se cumple la norma.
4. Registrar cada infracción en una base de datos SQLite (identificador, fecha y hora, tipo de falta y ruta de la captura) y guardar una fotografía JPG como evidencia.
5. Diseñar una interfaz gráfica con PyQt6 que incluya monitoreo en vivo con indicadores de estado, un historial consultable con vista previa y exportación a CSV, y una sección de ajustes configurables.
6. Verificar el funcionamiento mediante pruebas automáticas y gestionar el desarrollo de forma colaborativa con Git y GitHub.

3. Delimitación del proyecto

3.1 Alcance

El proyecto comprende el diseño y la construcción de una aplicación de escritorio que:

- Captura video de **una cámara web** conectada al computador.
- Detecta personas y verifica únicamente **casco y chaleco**.
- Emite alertas visuales y sonoras, y registra las infracciones con su captura en una base de datos local.
- Ofrece una interfaz con monitoreo en vivo, historial, exportación a CSV y ajustes persistentes (cámara, modelo, rendimiento, alarma, carpeta de capturas y tema).

3.2 Fuera del alcance

- Otros elementos de protección (guantes, gafas, calzado, arneses, protección auditiva).
- Múltiples cámaras, cámaras IP o transmisión de video por red.
- Identificación o reconocimiento de personas: el sistema registra infracciones, no identidades.
- Envío de notificaciones remotas (correo, mensajería), servicios en la nube o aplicación móvil.
- El entrenamiento de un modelo propio a gran escala. El sistema permite integrar un modelo de EPP ya entrenado, pero su entrenamiento no forma parte de esta fase.

3.3 Delimitación técnica, espacial y temporal

- **Técnica:** Python 3.10 o superior; ejecución solo en CPU; sistema operativo principal Windows 10/11.
- **Espacial:** las pruebas se realizan en un entorno controlado (aula o laboratorio) con la cámara web del equipo, simulando una zona de trabajo.
- **Temporal:** proyecto académico desarrollado durante el ciclo del curso; se entrega un prototipo inicial y su documentación.

3.4 Limitaciones conocidas

Con el modelo estándar (YOLOv8n entrenado en COCO) el sistema detecta personas, pero no reconoce cascos ni chalecos como objetos. Por ello, el prototipo comprueba el EPP mediante el **color** de la zona de la cabeza y del torso. Este método depende de la iluminación y del color del equipo, y puede producir falsos positivos o negativos; la precisión aumenta al usar un modelo entrenado en EPP, que el sistema detecta y utiliza automáticamente. El sistema es una ayuda para la supervisión y no la sustituye.

4. Justificación de las tecnologías a utilizar

La elección de tecnologías responde a tres criterios: que funcionen sin tarjeta gráfica, que sean de código abierto y bien documentadas, y que permitan trabajar de forma colaborativa y mantener el código ordenado.

Tecnología	Justificación
Python 3.10+	Lenguaje principal. Tiene el ecosistema más completo para visión por computadora y aprendizaje automático, es legible y facilita que todo el equipo colabore.
YOLOv8n (Ultralytics)	Detector de objetos en tiempo real. La variante "nano" es la más pequeña y rápida de la familia, adecuada para CPU. Además permite sustituirla por un modelo entrenado en EPP sin cambiar el resto del sistema.
OpenCV	Captura de video de la cámara web, preprocesamiento, análisis de color (HSV) y dibujo de recuadros y textos sobre el video.
NumPy	Manejo eficiente de los fotogramas como matrices; es base de OpenCV y de Ultralytics.
PyQt6	Interfaz gráfica de escritorio con aspecto profesional. Su sistema de hilos (QThread) y señales permite procesar video sin congelar la ventana, y sus hojas de estilo permiten un diseño moderno con tema claro y oscuro.

pygame-ce	Reproducción de la alarma sonora continua mediante su módulo de audio. Se usa la edición de la comunidad (pygame-ce), que publica instaladores precompilados para versiones recientes de Python y evita errores de compilación en Windows.
SQLite (sqlite3)	Base de datos incluida en la biblioteca estándar de Python. No necesita servidor ni configuración: todo se guarda en un único archivo (infracciones.db), ideal para una aplicación local.
Git y GitHub	Control de versiones y trabajo colaborativo mediante ramas, *commits* y *pull requests*, con historial verificable de la contribución de cada integrante.
unittest	Pruebas automáticas incluidas en Python, sin dependencias adicionales, para verificar la base de datos, las reglas de detección, la alarma y el flujo completo del monitoreo.

5. Requerimientos del proyecto

5.1 Requerimientos funcionales

Describen lo que el sistema debe hacer. La prioridad indica la importancia para la funcionalidad central del prototipo.

ID	Requerimiento	Prioridad
RF-01	Capturar video en tiempo real desde la cámara web a 640 x 480 píxeles.	Alta
RF-02	Detectar a las personas presentes en los fotogramas procesados mediante YOLOv8n.	Alta
RF-03	Verificar para cada persona la presencia de casco y chaleco, mediante reglas de color o un modelo entrenado en EPP.	Alta
RF-04	Marcar con un recuadro rojo y el texto "ALERTA: FALTA EPP" a la persona que incumpla.	Alta
RF-05	Marcar con un recuadro verde y el texto "EPP COMPLETO" a la persona que cumpla.	Alta
RF-06	Reproducir una alarma sonora continua mientras exista una infracción y detenerla cuando se regularice.	Alta
RF-07	Registrar cada infracción en SQLite (infracciones.db) con id, fecha y hora, tipo de falta y ruta de la captura.	Alta
RF-08	Guardar una captura JPG de cada infracción en la carpeta de capturas.	Alta
RF-09	Confirmar la infracción tras detecciones consecutivas y limitar la frecuencia de registros para evitar falsos positivos y duplicados.	Media

RF-10	Permitir iniciar y detener el monitoreo y silenciar la alarma desde la interfaz.	Alta
RF-11	Mostrar indicadores en tiempo real: estado general, cámara, personas, con y sin EPP, FPS, alarma, total de infracciones y actividad reciente.	Media
RF-12	Mostrar un historial de infracciones con fecha, tipo y foto, con búsqueda, filtro por tipo y vista previa de la captura.	Alta
RF-13	Exportar el historial a un archivo CSV.	Baja
RF-14	Permitir configurar y guardar ajustes: cámara, resolución, modelo, tamaño de imagen, confianza, salto de fotogramas, alarma, volumen, carpeta de capturas y tema.	Media
RF-15	Informar al usuario los fallos de cámara sin cerrar la aplicación.	Media

5.2 Requerimientos no funcionales

Describen las cualidades que debe tener el sistema. Las cifras son objetivos de diseño que se verificarán durante las pruebas.

ID	Requerimiento
RNF-01	**Rendimiento:** ejecutarse únicamente en CPU, con un objetivo de al menos 8 FPS en pantalla en un equipo de gama media, usando YOLOv8n, imgsz de 320 o 416 y salto de fotogramas.
RNF-02	**Tiempo de respuesta:** la alerta visual y sonora debe aparecer en menos de 2 segundos desde que ocurre la infracción, en condiciones normales.
RNF-03	**Usabilidad:** interfaz clara, con estados indicados por colores y textos, utilizable sin conocimientos técnicos y con tema claro y oscuro.
RNF-04	**Portabilidad:** funcionar en Windows 10/11 con Python 3.10 o superior, instalando las dependencias con *pip install -r requirements.txt*.
RNF-05	**Mantenibilidad:** arquitectura modular (núcleo, datos e interfaz separados), código comentado y pruebas automáticas.
RNF-06	**Fiabilidad:** el procesamiento de video no debe congelar la interfaz; si falla la cámara o el audio, el sistema informa el problema y continúa operando.
RNF-07	**Persistencia e integridad:** las infracciones se almacenan de forma local y duradera en SQLite; las rutas de las capturas se guardan de forma portable.
RNF-08	**Privacidad:** todo el procesamiento es local; no se envían imágenes ni datos a servicios externos.

RNF-09	**Colaboración:** el código se gestiona en un repositorio de GitHub con ramas y *pull requests*, con trazabilidad de los aportes de cada integrante.